

Introducción al Análisis de Seguridad

"Para garantizar la integridad del proyecto **webgame**, realizamos una validación técnica dividida en dos fases críticas: un **Modelado de Amenazas** teórico utilizando la librería **pytm** y un análisis práctico de seguridad estática o **SAST** mediante la herramienta **Snyk Code**. Esta metodología nos permitió corroborar que los riesgos identificados en el diseño estaban efectivamente presentes en nuestra base de código."

Parte 1: Modelado de Amenazas (Fundamento Teórico)

"Iniciamos mapeando la superficie de ataque entre el jugador, el servidor Express y la base de datos local. A través de este modelo, identificamos cuatro vectores de riesgo prioritarios:

1. **CR03 (Ataque de diccionario)**: Riesgo de compromiso de credenciales en el módulo de autenticación.
2. **INP31 (Inyección de comandos/SQL)**: Posible manipulación de la base de datos a través de los modelos de 'gallos' y usuarios.
3. **DS06 (Fuga de datos)**: Exposición de llaves de cifrado o datos de pasarelas de pago como Stripe.
4. **DE03 (Sniffing)**: Intercepción de tráfico por el uso de protocolos no cifrados en red local."

¿Por qué aparecen más de 100 amenazas?

La librería **pytm** funciona mediante un motor de inferencia basado en la taxonomía **STRIDE**. Cuando declaras un elemento (como un **Server** o un **Datastore**) y un flujo de datos (**Dataflow**), la herramienta genera automáticamente todas las amenazas que son **teóricamente posibles** para ese tipo de componente.

- **Es un catálogo exhaustivo**: El archivo **reporte_amenazas.md** actúa como un "diccionario de miedos". Incluye desde desbordamientos de búfer (INP01) hasta secuestro de sesiones (CR01).
- **Falsos positivos teóricos**: Muchas de esas 100 amenazas podrían no aplicar si el framework (Express) ya las maneja o si el entorno (local) las limita.

¿Por qué enfocarse solo en esas 4?

La transición del **Modelado de Amenazas (Teórico)** al **Análisis SAST (Práctico)** es la clave. Seleccionaron esas cuatro porque cumplen con tres criterios críticos de ingeniería:

1. **Correlación Directa (Validación)**: Fueron las amenazas que **Snyk Code** confirmó que existían realmente en el código fuente de **server.js** y **user.js**. Por ejemplo, el modelo predijo "Fuga de

datos" (DS06) y Snyk encontró la llave hardcodeada.

2. **Priorización por Riesgo:** Se eligieron los vectores que representaban el mayor peligro para el proyecto **WebGame**:
 - **CR03 y DS06:** Protegen el activo más valioso (las credenciales y pagos de los jugadores).
 - **INP31:** Evita la destrucción de la base de datos.
 - **DE03:** Es la vulnerabilidad base de cualquier sistema que viaja por red.
3. **Viabilidad de Remediación:** Son puntos donde una acción técnica clara (como cambiar a **AES-256-GCM** o usar **Helmet**) elimina el riesgo de forma inmediata y medible.

Parte 2: Análisis SAST y Resultados (Evidencia Práctica)

"Al ejecutar el escaneo con **Snyk**, detectamos **5 vulnerabilidades** : una de severidad **Alta** y cuatro **Medias**. Los puntos críticos se localizaron en `server.js` y, principalmente, en `src/models/user.js`.

Los hallazgos principales fueron:

- **Hardcoded Secret (Severidad Alta):** El uso de una llave de cifrado definida directamente en el código fuente (línea 63 de `user.js`), lo cual facilita la exposición de datos sensibles.
- **Cifrado sin protección de integridad:** El uso de `crypto.createCipher` con el modo Counter, que no garantiza que los datos no hayan sido alterados.
- **Transmisión en texto plano:** El servidor utilizaba el módulo `http`, exponiendo la comunicación a ataques de interceptación.
- **Exposición de información y falta de límites:** El framework revelaba su identidad mediante la cabecera `X-Powered-By` y carecía de mecanismos de *rate-limiting* en operaciones costosas del sistema de archivos."

Open Issues

X [MEDIUM] Information Exposure - X-Powered-By Header

Finding ID: 1314e5cf-4ad2-437f-8084-783d566effb9

Path: server.js, line 11

Info: Disable X-Powered-By header for your Express app (consider using Helmet middleware), because it exposes information about the used framework to potential attackers.

X [MEDIUM] Cleartext Transmission - HTTP Instead of HTTPS

Finding ID: a6f1d01b-8981-4461-a5c9-adebc7738ad3

Path: server.js, line 165

Info: http.createServer uses HTTP which is an insecure protocol and should not be used in code due to cleartext transmission of information. Data in cleartext in a communication channel can be sniffed by unauthorized actors. Consider using the https module instead.

X [MEDIUM] Use of Cipher Without Integrity Protection

Finding ID: f43bd474-48a7-49a8-864b-b0320e622d8b

Path: src/models/user.js, line 63

Info: Counter mode (used in crypto.createCipher) does not provide integrity. Consider using Galois/Counter mode.

X [MEDIUM] Allocation of Resources Without Limits or Throttling

Finding ID: 2569f0c5-d951-4d98-be59-9cf33dbb55f4

Path: server.js, line 159

Info: Expensive operation (a file system operation) is performed by an endpoint handler which does not use a rate-limiting mechanism. It may enable the attackers to perform Denial-of-service attacks. Consider using a rate-limiting middleware such as express-limit.

X [HIGH] Hardcoded Secret

Finding ID: 83ad7ac1-f2f5-472d-8a24-029d78bf2b8c

Path: src/models/user.js, line 63

Info: Hardcoded value is used as a cipher key (in crypto.createCipher). Generate the value with a cryptographically strong random number generator and do not hardcode it in source code.

Test Summary

Organization: elizan626

Test type: Static code analysis

Project path: C:\Users\elija\OneDrive\Escritorio\Programing\6to-Semestre\Soft-Seguro\WebGame\BACKEND

Total issues: 5

Ignored issues: 0 [0 HIGH 0 MEDIUM 0 LOW]

Open issues: 5 [1 HIGH 4 MEDIUM 0 LOW]

Parte 3: Implementación de Mitigaciones (El "Antes y Después")

"Basándonos en estos resultados, aplicamos las siguientes recomendaciones técnicas que pueden observar en las capturas de código:

1. **Gestión de Secretos:** Eliminamos la llave hardcoded. Ahora, como ven en la captura corregida, el sistema utiliza process.env.CIPHER_KEY para cargar la credencial desde un entorno seguro no indexado en el control de versiones.
2. **Robustez Criptográfica:** Migramos el motor de cifrado. Pasamos de un método inseguro a crypto.createCipheriv utilizando el algoritmo **AES-256-GCM**. Esto no solo cifra la información, sino que añade un 'Authentication Tag' (como se observa en el código con getAuthTag), garantizando la integridad de los datos de los usuarios.
3. **Endurecimiento del Servidor:**
 - Sustituimos el módulo http por https con certificados para el entorno local.

- Integramos el middleware Helmet para ocultar las cabeceras del framework y evitar el fingerprinting.
- Implementamos express-rate-limit para mitigar intentos de denegación de servicio (DoS) por saturación de peticiones."

Conclusión

"La ejecución de este plan de resolución transforma un prototipo vulnerable en una arquitectura robusta. Hemos pasado de una validación teórica a una mitigación práctica, asegurando que el código sea seguro desde su origen antes de cualquier despliegue en infraestructura de producción como AWS."